AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm drzewa BST
z zastosowaniem GitHub

Autor:
Damian Kiełbasa
Patryk Gargula

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **4**

1.1. Cel projektu 4
1.2. Wymagania funkcjonalne 4
1.3. Wymagania dotyczące dokumentacji i kontroli wersji 4

2. Analiza problemu **6**

2.1. Drzewo binarne poszukiwań i jego zastosowania 6
2.2. Sposób działania 6
2.2.0.1. Dodawanie. 7
2.2.0.2. Usuwanie. 7
2.2.0.3. Przechodzenie drzewa. 7
2.3. Git i GitHub 7

3. Projektowanie **8**

3.1. Narzędzia i środowisko 8
3.2. Struktura projektu 9
3.3. Podział pracy 9
3.4. Sposób używania systemu Git 9
3.4.0.1. Drzewo commitów. 10

4. Implementacja **10**

4.1. Struktura Node 10
4.2. Klasa BST 11
4.2.1. Dodawanie elementu 12
4.2.2. Usuwanie elementu 12
4.2.3. Wyświetlanie drzewa 13
4.3. Funkcja main i menu programu 14
4.4. Przykład działania 15
4.5. Konflikty przy scalaniu 16
4.6. Elementy w przygotowaniu 16

5. Wnioski **17**

Literatura **18**

Spis rysunków **18**

Spis tabel **19**

Spis listingów **20**

1. Ogólne określenie wymagań

1.1. Cel projektu

Celem projektu jest napisanie w języku C++ programu realizującego drzewo binarne poszukiwań (BST, ang. *Binary Search Tree*) z wykorzystaniem kontenerów biblioteki STL. Drzewo ma zostać zaimplementowane w klasie, a każda klasa ma znajdować się w osobnych plikach, podobnie jak funkcja `main`. Program ma udostępniać użytkownikowi menu, w którym wybiera on kolejne operacje na drzewie oraz operacje zapisu i odczytu plików.

Drugim celem, równie ważnym jak sam program, jest poznanie możliwości serwisu GitHub w pracy nad projektem grupowym. Projekt jest realizowany w dwuosobowym zespole, a przebieg pracy (commity, gałęzie, scalanie, konflikty) ma zostać pokazany w historii repozytorium. Niniejszy dokument opisuje zarówno program, jak i sposób wykorzystania tych narzędzi.

Przewidywane rezultaty projektu to:

- działający program z kompletem metod drzewa BST,
- klasa realizująca zapis i odczyt drzewa z pliku binarnego oraz wczytywanie liczb z pliku tekstowego,
- repozytorium na GitHubie z historią zmian dwóch autorów,
- niniejsza dokumentacja projektowa przygotowana w LaTeXu.

1.2. Wymagania funkcjonalne

Drzewo BST ma udostępniać operacje wymienione w tabeli 1.1. W tabeli podano również nazwy metod, którymi zrealizowano poszczególne wymagania, oraz aktualny stan prac.

Program ma ponadto posiadać w funkcji `main` menu z opcjami drzewa oraz odczytu i zapisu pliku. Program czeka na wybranie opcji, a użytkownik może go poprawnie zamknąć specjalną opcją wyjścia.

1.3. Wymagania dotyczące dokumentacji i kontroli wersji

Podczas realizacji projektu należy:

- utworzyć nowy projekt na GitHubie realizowany w dwuosobowej grupie,

Tab. 1.1. Wymagania funkcjonalne i ich realizacja

Lp.	Wymaganie	Metoda	Stan
1	Dodaj element	`insert`	zrealizowane
2	Usuń element	`remove`	zrealizowane
3	Usuń całe drzewo	nazwa metody	w trakcie prac
4	Szukaj drogi do podanego elementu	nazwa metody	w trakcie prac
5	Wyświetl drzewo (preorder, inorder, postorder)	`display`	zrealizowane
6	Wyświetl drzewo graficznie na ekranie	nazwa metody	w trakcie prac
7	Zapis drzewa do pliku tekstowego	nazwa metody	w trakcie prac
8	Zapis drzewa do pliku binarnego i odczyt z niego	klasa i metody	w trakcie prac
9	Wczytanie liczb z pliku tekstowego do pustego lub istniejącego drzewa	nazwa metody	w trakcie prac

- wykonać co najmniej 5 commitów każdej z osób,
- pokazać pracę, w której najpierw jedna osoba tworzy gałąź i po kilku commitach ją scala, a następnie druga osoba tworzy gałąź i po kilku commitach scala ją do gałęzi głównej,
- pokazać pracę równoległą, w której obie osoby tworzą nową gałąź (w jednym punkcie zaczynają pracę równolegle), a po kilku commitach wykonują scalenie do gałęzi głównej,
- rozwiązać co najmniej 6 konfliktów przy scalaniu gałęzi (3 jedna osoba, 3 druga osoba),
- dołączyć do projektu dokumentację w LaTeXu (niniejszy dokument) z obrazem drzewa commitów.

Stopień realizacji tych wymagań w aktualnej wersji projektu opisano w rozdziale 3.4.

2. Analiza problemu

2.1. Drzewo binarne poszukiwań i jego zastosowania

Drzewo binarne poszukiwań jest dynamiczną strukturą danych złożoną z węzłów[1]. Każdy węzeł przechowuje wartość oraz dwa wskaźniki: na lewe i na prawe poddrzewo. Węzeł, który nie ma potomka, ma w odpowiednim wskaźniku wartość `nullptr`. Najwyżej położony węzeł nazywa się korzeniem. Cechą wyróżniającą drzewo BST jest porządek wartości: w lewym poddrzewie każdego węzła znajdują się wyłącznie wartości mniejsze od jego wartości, a w prawym wyłącznie większe. Dzięki temu przy wyszukiwaniu w każdym kroku można odrzucić jedno z poddrzew.

Drzewa BST i ich pochodne stosuje się między innymi w:

- słownikach i zbiorach uporządkowanych (np. `std::set` i `std::map` są zwykle implementowane jako zrównoważone drzewa poszukiwań),
- indeksach w bazach danych,
- wyszukiwaniu i sortowaniu danych (przejście *inorder* daje wartości w porządku rosnącym),
- analizie wyrażeń i kompilatorach.

2.2. Sposób działania

Klasa drzewa przechowuje jeden wskaźnik `root` na korzeń. Puste drzewo to takie, w którym `root` jest równy `nullptr`. Przykładowe drzewo o siedmiu elementach przedstawia rysunek 2.1.

[level distance=1.2cm, level 1/.style=sibling distance=4.6cm, level 2/.style=sibling distance=2.3cm, every node/.style=circle,draw,minimum size=0.9cm] 50 child node 30 child node 20 child node 40 child node 70 child node 60 child node 80 ;

Rys. 2.1. Przykładowe drzewo BST utworzone przez wstawienie wartości 50, 30, 70, 20, 40, 60, 80

[1] Podstawy struktur danych: [**cormen**].

2.2.0.1. Dodawanie. Wartość porównuje się z wartością bieżącego węzła, zaczynając od korzenia. Mniejsza wartość trafia do lewego poddrzewa, większa do prawego. Gdy napotkany zostanie pusty wskaźnik, w jego miejscu tworzony jest nowy węzeł. Wartość, która już występuje w drzewie, nie jest dodawana ponownie.

2.2.0.2. Usuwanie. Po odnalezieniu węzła rozróżnia się trzy przypadki: węzeł bez potomków jest po prostu usuwany, węzeł z jednym potomkiem jest zastępowany tym potomkiem, a węzeł z dwoma potomkami otrzymuje wartość swojego następnika (najmniejszej wartości w prawym poddrzewie), po czym ten następnik jest usuwany z prawego poddrzewa.

2.2.0.3. Przechodzenie drzewa. Istnieją trzy podstawowe sposoby odwiedzenia wszystkich węzłów (tabela 2.1). Dla drzewa z rysunku 2.1 dają one różne ciągi wartości.

Tab. 2.1. Metody przechodzenia drzewa

Metoda	**Kolejność odwiedzania**	**Wynik dla rys. 2.1**
preorder	węzeł, lewe, prawe	50 30 20 40 70 60 80
inorder	lewe, węzeł, prawe	20 30 40 50 60 70 80
postorder	lewe, prawe, węzeł	20 40 30 60 80 70 50

Złożoność operacji zależy od wysokości drzewa h (tabela 2.2). Dla drzewa zrównoważonego $h = O(\log n)$, ale gdy wartości są dodawane w porządku rosnącym lub malejącym, drzewo zamienia się w listę i $h = n$. Zwykłe drzewo BST nie zapewnia samo równoważenia.

Tab. 2.2. Złożoność czasowa operacji na drzewie BST

Operacja	**Przypadek średni**	**Najgorszy**	**Uzasadnienie**
dodawanie, usuwanie, szukanie	$O(\log n)$	$O(n)$	zejście o h poziomów
przechodzenie (pre-, in-, postorder)	$O(n)$	$O(n)$	odwiedzenie każdego
usunięcie całego drzewa	$O(n)$	$O(n)$	usunięcie każdego węz

2.3. Git i GitHub

Git to rozproszony system kontroli wersji[2]. Zapamiętuje kolejne stany projektu w postaci commitów, czyli migawek plików wraz z opisem zmian, autorem i datą.

[2]Opis systemu Git: [**progit**].

Główne pojęcia to:

- repozytorium – katalog projektu wraz z historią zmian,
- commit – zapisany w historii stan projektu,
- gałąź (*branch*) – równoległa linia rozwoju projektu,
- scalanie (*merge*) – połączenie zmian z dwóch gałęzi w jedną,
- konflikt – sytuacja, w której obie scalane gałęzie zmieniły to samo miejsce w pliku i Git nie potrafi samodzielnie wybrać wersji,
- *push* i *pull* – wysłanie zmian na serwer i pobranie zmian z serwera.

GitHub[3] jest serwisem internetowym hostującym repozytoria Git. Pełni rolę wspólnego, zdalnego repozytorium, dzięki któremu dwie osoby mogą równolegle pracować nad tym samym kodem. Przy pracy równoległej konflikty są nieuniknione, dlatego ważne jest zrozumienie, jak wygląda plik po nieudanym scaleniu: Git oznacza w nim sporne fragmenty znacznikami `<<<<<<<`, `=======` i `>>>>>>>`, a osoba scalająca musi wybrać lub połączyć obie wersje i dokończyć scalanie commitem.

3. Projektowanie

3.1. Narzędzia i środowisko

Program napisano w języku C++[4]. Wymagany jest kompilator obsługujący standard C++11 (w kodzie używane jest słowo kluczowe `nullptr`). Obecna wersja programu korzysta wyłącznie z nagłówka `<iostream>`; kontenery STL zostaną użyte w dalszej części prac (zapis i odczyt plików).

Narzędzia wykorzystane w projekcie:

- Visual Studio 2022,
- Git i GitHub – kontrola wersji i zdalne repozytorium,
- Overleaf.

[3]Dokumentacja serwisu: [**ghdocs**].
[4]Opis języka: [**stroustrup**], [**cppref**].

3.2. Struktura projektu

Zgodnie z wymaganiem klasy i funkcja `main` znajdują się w oddzielnych plikach (tabela 3.1).

Tab. 3.1. Pliki projektu

Plik	**Zawartość**
`BST.h`, `BST.cpp`	struktura `Node` i klasa `BST` – drzewo binarne poszukiwań
`main.cpp`	funkcja `main` z menu programu
pliki klasy zapisu/odczytu	druga klasa: zapis i odczyt drzewa z pliku binarnego oraz wczytanie liczb z pliku tekstowego (w przygotowaniu)
`.gitignore`	lista plików pomijanych przez Git

3.3. Podział pracy

Pracę podzielono między dwóch autorów (tabela 3.2).

Tab. 3.2. Podział pracy w zespole

Autor	**Zakres prac**
Damian Kiełbasa	utworzenie szkieletu projektu i początkowych plików (`BST.h`, `BST.cpp`, `main.cpp`); dodawanie elementów (`insert`); pełna logika usuwania węzłów (`remove`, `deleteRec`, `minValueNode`); aktualizacja menu głównego
Patryk Gargula	metody wyświetlania drzewa na trzy sposoby (preorder, inorder, postorder); obsługa wyświetlania w menu w funkcji `main`; scalenie (*merge*) pracy równoległej i rozwiązanie pierwszego konfliktu w repozytorium

3.4. Sposób używania systemu Git

Praca nad projektem przebiega w cyklu: zmiana kodu, sprawdzenie, czy program się kompiluje, dodanie zmian do poczekalni (`git add`), zapisanie ich w historii (`git commit`) i wysłanie na GitHuba (`git push`). Commit obejmuje jedną niewielką

zmianę (np. dodanie jednej metody), a jego opis mówi, co zostało zmienione. Polecenia potrzebne do pracy na gałęziach i scalania przedstawia listing 1.

```
git checkout -b wyswietlanie   # utworzenie galezi i przejscie na
    nia
git add .                      # dodanie plikow do poczekalni
git commit -m "Dodano printInorder"
git push -u origin wyswietlanie
git checkout main              # powrot na galaz glowna
git merge wyswietlanie         # scalenie galezi do main
git log --oneline --graph      # historia commitow w postaci grafu
```

Listing 1. Polecenia Git użyte przy pracy na gałęziach

Stan realizacji wymagań dotyczących pracy z GitHubem zebrano w tabeli 3.3.

Tab. 3.3. Stan realizacji wymagań dotyczących GitHuba

Wymaganie	**Stan**	**Uwagi**
Co najmniej 5 commitów każdej osoby	w trakcie	liczba commitów
Gałąź jednej osoby scalona do głównej	zrealizowane częściowo	nazwa gałęzi
Gałąź drugiej osoby scalona do głównej	w trakcie	nazwa gałęzi
Praca równoległa obu osób na nowej gałęzi i scalenie	w trakcie	nazwa gałęzi
6 konfliktów (3 + 3)	rozwiązany 1 z 6	patrz rozdz. 4.5

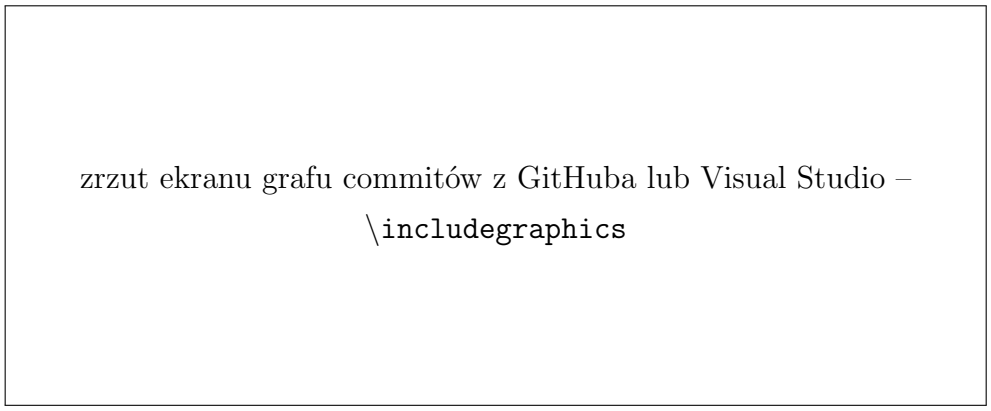
3.4.0.1. Drzewo commitów. Obraz drzewa commitów z repozytorium przedstawia rysunek 3.1.

4. Implementacja

4.1. Struktura Node

Struktura `Node` (listing 2) reprezentuje węzeł drzewa. Przechowuje wartość całkowitą oraz wskaźniki na lewe i prawe poddrzewo. Konstruktor ustawia wartość i zeruje oba wskaźniki.

```
struct Node {
    int data;
```



Rys. 3.1. Drzewo commitów w repozytorium projektu

```
    Node* left;
    Node* right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};
```

Listing 2. Struktura Node

4.2. Klasa BST

Klasa `BST` (listing 3) przechowuje wskaźnik `root` na korzeń. Metody publiczne stanowią interfejs dla użytkownika, natomiast operacje rekurencyjne są ukryte w sekcji prywatnej. Dzięki temu kod korzystający z klasy nie musi znać wewnętrznej budowy drzewa ani przekazywać wskaźnika na korzeń.

```
class BST {
private:
    Node* root;

    Node* insertRec(Node* node, int data);
    Node* deleteRec(Node* node, int data);
    Node* minValueNode(Node* node);

    void printPreorder(Node* node);
    void printInorder(Node* node);
    void printPostorder(Node* node);

public:
    BST();
    void insert(int data);
    void remove(int data);
    void display(int method);
```

```
18 };
```

Listing 3. Deklaracja klasy BST

4.2.1. Dodawanie elementu

Metoda `insert` (listing 4) wywołuje rekurencyjną metodę prywatną `insertRec`. Gdy napotka pusty wskaźnik, tworzy w tym miejscu nowy węzeł. W przeciwnym razie schodzi do lewego lub prawego poddrzewa, zależnie od porównania wartości. Jeśli wartość już występuje w drzewie, nie jest dodawana ponownie, a metoda zwraca niezmieniony węzeł.

```
1  void BST::insert(int data) {
2      root = insertRec(root, data);
3  }
4
5  Node* BST::insertRec(Node* node, int data) {
6      if (node == nullptr) return new Node(data);
7      if (data < node->data) node->left = insertRec(node->left, data)
       ;
8      else if (data > node->data) node->right = insertRec(node->right
       , data);
9      return node;
10 }
```

Listing 4. Metody insert i insertRec

4.2.2. Usuwanie elementu

Metoda `remove` (listing 5) przekazuje usuwanie do `deleteRec`. Po odnalezieniu węzła obsługiwane są trzy przypadki opisane w rozdziale 2. Gdy brakuje lewego lub prawego potomka, węzeł jest zastępowany drugim potomkiem (lub `nullptr`), a jego pamięć zwalniana. Gdy istnieją obaj potomkowie, metoda `minValueNode` wyszukuje następnik w prawym poddrzewie, jego wartość jest kopiowana do usuwanego węzła, a następnie następnik jest usuwany z prawego poddrzewa. Usunięcie wartości, której nie ma w drzewie, nie zmienia drzewa.

```
1  void BST::remove(int data) {
2      root = deleteRec(root, data);
3  }
4
5  Node* BST::minValueNode(Node* node) {
6      Node* current = node;
7      while (current && current->left != nullptr)
```

```
8           current = current->left;
9       return current;
10  }
11
12  Node* BST::deleteRec(Node* node, int data) {
13      if (node == nullptr) return node;
14
15      if (data < node->data) node->left = deleteRec(node->left, data)
        ;
16      else if (data > node->data) node->right = deleteRec(node->right
        , data);
17      else {
18          if (node->left == nullptr) {
19              Node* temp = node->right;
20              delete node;
21              return temp;
22          } else if (node->right == nullptr) {
23              Node* temp = node->left;
24              delete node;
25              return temp;
26          }
27          Node* temp = minValueNode(node->right);
28          node->data = temp->data;
29          node->right = deleteRec(node->right, temp->data);
30      }
31      return node;
32  }
```

Listing 5. Metody remove, deleteRec i minValueNode

4.2.3. Wyświetlanie drzewa

Metoda `display` (listing 6) przyjmuje numer metody przechodzenia: 1 – preorder, 2 – inorder, 3 – postorder. Dla pustego drzewa wypisuje komunikat, a dla nieznanego numeru informację o nieznanej metodzie. Właściwe przejście wykonują trzy rekurencyjne metody prywatne, które różnią się wyłącznie miejscem wypisania wartości węzła względem wywołań dla poddrzew.

```
1   void BST::display(int method) {
2       if (root == nullptr) {
3           cout << "Drzewo jest puste.\n";
4           return;
5       }
6       switch (method) {
7           case 1: printPreorder(root); break;
```

```
        case 2: printInorder(root); break;
        case 3: printPostorder(root); break;
        default: cout << "Nieznana metoda.\n";
    }
    cout << endl;
}

void BST::printPreorder(Node* node) {
    if (node == nullptr) return;
    cout << node->data << " ";
    printPreorder(node->left);
    printPreorder(node->right);
}

void BST::printInorder(Node* node) {
    if (node == nullptr) return;
    printInorder(node->left);
    cout << node->data << " ";
    printInorder(node->right);
}

void BST::printPostorder(Node* node) {
    if (node == nullptr) return;
    printPostorder(node->left);
    printPostorder(node->right);
    cout << node->data << " ";
}
```

Listing 6. Metoda display i metody przechodzenia drzewa

4.3. Funkcja main i menu programu

Funkcja `main` (listing 7) znajduje się w osobnym pliku. W pętli wyświetla menu i wykonuje wybraną opcję, aż użytkownik wybierze 0, co kończy program. Numeracja opcji pozostawia wolne numery dla funkcji, które zostaną dodane w kolejnych etapach (m.in. usuwanie całego drzewa, szukanie drogi, zapis i odczyt plików).

```
void printMenu() {
    cout << "\n--- MENU ---\n";
    cout << "1. Dodaj element\n";
    cout << "2. Usun element z drzewa\n";
    cout << "5. Wyswietl drzewo\n";
    cout << "0. Wyjscie\n";
    cout << "Wybierz opcje: ";
```

```
}

int main() {
    BST tree;
    int choice, val;

    do {
        printMenu();
        cin >> choice;

        if (choice == 1) {
            cout << "Podaj wartosc: ";
            cin >> val;
            tree.insert(val);
        }
        else if (choice == 2) {
            cout << "Podaj wartosc do usuniecia: ";
            cin >> val;
            tree.remove(val);
        }
        else if (choice == 5) {
            int method;
            cout << "Metoda (1-pre, 2-in, 3-post): ";
            cin >> method;
            tree.display(method);
        }
    } while (choice != 0);

    return 0;
}
```

Listing 7. Funkcja main i menu

4.4. Przykład działania

Po dodaniu wartości 50, 30, 70, 20, 40, 60 i 80 (drzewo z rysunku 2.1) wybranie opcji 5 daje wyniki zgodne z tabelą 2.1: dla metody 1 ciąg `50 30 20 40 70 60 80`, dla metody 2 ciąg `20 30 40 50 60 70 80`, a dla metody 3 ciąg `20 40 30 60 80 70 50`. zrzut ekranu lub wynik uruchomienia programu

4.5. Konflikty przy scalaniu

Pierwszy konflikt powstał podczas scalania równoległej pracy obu autorów. Damian zmodyfikował funkcję `printMenu` w pliku `main.cpp` w ramach prac nad usuwaniem elementów, a Patryk w tym samym miejscu dodał do menu opcję wyświetlania drzewa. Ponieważ obie zmiany dotyczyły tej samej linijki menu, Git nie mógł ich scalić automatycznie i oznaczył spór znacznikami (listing 8). Konflikt rozwiązał Patryk, zachowując w menu obie opcje, a następnie zatwierdził scalenie commitem.

```
<<<<<<< HEAD
    cout << "2. Usun element z drzewa\n";
=======
    cout << "5. Wyswietl drzewo\n";
>>>>>>> wyswietlanie
```

Listing 8. Schematyczna postać pliku main.cpp w trakcie konfliktu

Zestawienie wszystkich konfliktów wymaganych w zadaniu przedstawia tabela 4.1.

Tab. 4.1. Konflikty rozwiązane podczas scalania gałęzi

Nr	Rozwiązał	Plik i miejsce	Opis
1	Patryk	`main.cpp`, funkcja `printMenu`	jednoczesna edycja tej samej linijki menu; zachowano obie opcje
2	osoba	plik	opis
3	osoba	plik	opis
4	osoba	plik	opis
5	osoba	plik	opis
6	osoba	plik	opis

4.6. Elementy w przygotowaniu

Do ukończenia projektu pozostało zaimplementowanie: usuwania całego drzewa, szukania drogi do podanego elementu, graficznego wyświetlania drzewa na ekranie, zapisu do pliku tekstowego, drugiej klasy odpowiedzialnej za zapis i odczyt drzewa w pliku binarnym oraz wczytywania liczb z pliku tekstowego do pustego lub istniejącego drzewa. opis po implementacji

5. Wnioski

W ramach dotychczasowych prac zaimplementowano w języku C++ drzewo binarne poszukiwań w klasie `BST`, z operacjami dodawania i usuwania elementów oraz wyświetlania drzewa metodami preorder, inorder i postorder. Klasa, struktura węzła i funkcja `main` znajdują się w oddzielnych plikach, a program udostępnia menu, które pozwala poprawnie zamknąć aplikację. Pozostałe wymagania (usuwanie całego drzewa, szukanie drogi, wyświetlanie graficzne, zapis i odczyt plików) są w trakcie realizacji. uzupełnić po zakończeniu projektu

Implementacja potwierdziła zalety drzewa BST: w drzewie zrównoważonym dodawanie, usuwanie i szukanie wymagają średnio czasu logarytmicznego, a przejście *inorder* daje wartości w porządku rosnącym. Ceną jest brak gwarancji zrównoważenia (po dodaniu wartości posortowanych drzewo staje się listą o złożoności liniowej) oraz konieczność ręcznego pilnowania wskaźników. Najwięcej uwagi wymagało usuwanie węzła z dwoma potomkami.

Podczas analizy zauważono ograniczenia obecnej wersji, które zostaną usunięte w dalszych pracach:

- klasa `BST` nie ma destruktora, więc pamięć zajmowana przez węzły nie jest zwalniana przy zniszczeniu obiektu; zostanie to rozwiązane przez metodę usuwającą całe drzewo wywoływaną w destruktorze,
- program nie sprawdza poprawności danych wpisywanych w menu; wpisanie znaku zamiast liczby powoduje błąd strumienia wejściowego i może zablokować pętlę menu,
- metody rekurencyjne dla bardzo głębokiego (zdegenerowanego) drzewa mogą wyczerpać stos wywołań,
- drzewo przechowuje tylko wartości typu `int`; uogólnienie można uzyskać przez zastosowanie szablonów (`template`),
- wartości powtarzające się są ignorowane, bez informowania o tym użytkownika.

Praca w zespole z wykorzystaniem Gita i GitHuba pozwoliła podzielić zadania między autorów i prześledzić historię zmian. Konflikt powstały przy scalaniu gałęzi, spowodowany jednoczesną edycją tej samej linijki menu, pokazał, że przy pracy równoległej na tym samym pliku warto często synchronizować gałęzie i dzielić kod tak, aby autorzy rzadziej modyfikowali te same fragmenty. wnioski dotyczące pozostałych konfliktów i scalania gałęzi

Spis rysunków

2.1. Przykładowe drzewo BST utworzone przez wstawienie wartości 50, 30, 70, 20, 40, 60, 80 6

3.1. Drzewo commitów w repozytorium projektu 11

Spis tabel

1.1. Wymagania funkcjonalne i ich realizacja 5
2.1. Metody przechodzenia drzewa 7
2.2. Złożoność czasowa operacji na drzewie BST 7
3.1. Pliki projektu 9
3.2. Podział pracy w zespole 9
3.3. Stan realizacji wymagań dotyczących GitHuba 10
4.1. Konflikty rozwiązane podczas scalania gałęzi 16

Spis listingów

1. Polecenia Git użyte przy pracy na gałęziach 10
2. Struktura Node 10
3. Deklaracja klasy BST 11
4. Metody insert i insertRec 12
5. Metody remove, deleteRec i minValueNode 12
6. Metoda display i metody przechodzenia drzewa 13
7. Funkcja main i menu 14
8. Schematyczna postać pliku main.cpp w trakcie konfliktu 16